

Atividade Prática: Biblioteca Pessoal (Minha Estante)

Objetivo

Desenvolver uma aplicação Web completa utilizando **Node.js**, **Express**, **EJS** e o padrão de arquitetura **MVC (Model-View-Controller)**, com persistência de dados em arquivo **JSON** e estilização com **CSS** (bootstrap ou tailwind, à sua escolha).

O objetivo desta atividade é fixar os conceitos aprendidos no projeto ToDoList, criando um sistema para gerenciar leituras pessoais com organização modular de código e uso correto de HTML5 semântico.

Estrutura Esperada do Projeto

text

```
minha-estante/  
├── controllers/  
│   └── livroController.js  
├── models/  
│   └── livroModel.js  
├── routes/  
│   └── livroRoutes.js  
├── views/  
│   ├── partials/  
│   │   ├── header.ejs  
│   │   └── footer.ejs  
│   ├── index.ejs  
│   ├── principal.ejs  
│   └── novoLivro.ejs  
├── livros.json  
├── app.js  
├── package.json  
└── .gitignore
```

Passo a Passo de Configuração

1. Inicialização do Projeto (Terminal)

```
bash
```

```
npm init -y  
npm install express ejs
```

2. Configuração do `package.json`

Adicionar `"type": "module"` para utilizar a sintaxe moderna do ES Modules (`import/export`) e os scripts de inicialização:

```
json
```

```
{  
  "main": "app.js",  
  "type": "module",  
  "scripts": {  
    "start": "node app.js",  
    "dev": "node --watch app.js"  
  }  
}
```

3. Criar a base de dados (`livros.json`)

Crie o arquivo `livros.json` na raiz com alguns registros de exemplo:

```
json
```

```
[  
  {  
    "titulo": "Dom Casmurro",  
    "autor": "Machado de Assis",  
    "paginas": 256,  
    "status": "Lido"  
  },  
  {  
    "titulo": "O Hobbit",  
    "autor": "J.R.R. Tolkien",  
    "paginas": 310,  
    "status": "Lendo"  
  }  
]
```

Regras de Negócio e Arquitetura MVC

1. Camada Model (`models/livroModel.js`)

Responsável pelo acesso e persistência no arquivo `livros.json` com o módulo nativo `fs` :

- `carregaLivros()` : Lê o arquivo com `fs.readFileSync` e devolve o array de livros (ou `[]` caso haja erro).
- `salvarLivros(livros)` : Escreve no arquivo com `fs.writeFileSync` formatado (`JSON.stringify(livros, null, 2)`).
- `adicionarLivro(novoLivro)` : Inclui um novo livro no array e salva no arquivo.
- `alterarStatus(indice)` : Altera o status do livro no índice indicado ("*Lendo*" ↔ "*Lido*") e salva.
- `removerLivro(indice)` : Remove o livro do array utilizando `.splice(indice, 1)` e salva.

2. Camada Controller (`controllers/livroController.js`)

Faz a mediação entre as requisições HTTP e as operações do Model:

- `listar(req, res)` : Busca os livros via Model e renderiza a tela `principal.ejs`.
- `formNovo(req, res)` : Renderiza a página do formulário em `novoLivro.ejs`.
- `criar(req, res)` : Lê os dados enviados no corpo da requisição (`req.body`), define o status inicial como "*Lendo*", chama `adicionarLivro()` do Model e redireciona para `/livros`.
- `alterarStatus(req, res)` : Obtém o parâmetro `indice` da URL, aciona a troca de status no Model e redireciona para `/livros`.
- `remover(req, res)` : Obtém o parâmetro `indice` da URL, chama `removerLivro()` no Model e redireciona para `/livros`.

3. Rotas com `express.Router` (`routes/livroRoutes.js`)

Modularizar os endpoints associados ao recurso `/livros` :

- `GET /` → `livroController.listar`
- `GET /novo` → `livroController.formNovo`
- `POST /novo` → `livroController.criar`
- `GET /status/:indice` → `livroController.alterarStatus`
- `GET /remover/:indice` → `livroController.remover`

4. Arquivo Principal (`app.js`)

- Configurar o motor de templates: `app.set("view engine", "ejs")`.
- Habilitar middleware para dados de formulários: `app.use(express.urlencoded({ extended: true }))`.

- Rota raiz de apresentação:
 - `GET /` → Renderiza a página inicial `index.ejs`.
 - Conectar o roteador modular:
 - `app.use("/livros", livroRoutes)`.
 - Iniciar o servidor na porta `8080` (exibindo mensagem informativa no console).
-

Views e Interface com EJS (HTML5 Semântico)

1. Partial (`views/partials/`)

- `header.ejs` :
 - Tag `<head>` com CDN do Tailwind CSS.
 - Tag `<header>` com o nome da aplicação ("Minha Estante") e menu de navegação `<nav>` com links para `/` (Início), `/livros` (Meus Livros) e `/livros/novo` (Cadastrar Livro).
 - Abertura da tag principal: `<main class="container mx-auto p-4">`.
- `footer.ejs` :
 - Fechamento da tag `</main>`.
 - Rodapé semântico com `<footer>` e texto de identificação em `<small>`.

2. Páginas (`views/`)

- `index.ejs` :
 - Apresentação da aplicação usando `<article>`, banner de boas-vindas e botão de acesso rápido à estante.
 - `principal.ejs` :
 - Tabela ou cards estilizados listando os livros cadastrados.
 - Informações: Título, Autor, Páginas, Status e Ações.
 - Ações:
 - Botão/link para alternar status (ex: "Marcar como Lido" / "Marcar como Lendo").
 - Botão/link para remoção direcionando para `/livros/remover/<%= index %>`.
 - `novoLivro.ejs` :
 - Formulário organizado com tags semânticas: `<section>`, `<fieldset>`, `<legend>`, `<label>`, `<input>` e `<menu>` para botões.
 - Campos:
 - Título do Livro (`type="text"`, `required`)
 - Autor (`type="text"`, `required`)
 - Quantidade de Páginas (`type="number"`, `min="1"`, `required`)
-

Boas Práticas e Requisitos de Qualidade

1. **HTML5 Semântico:** Evitar o uso excessivo de `<div>` . Priorizar: `<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<figure>` , `<hgroup>` , `<fieldset>` , `<menu>` e `<footer>` .
 2. **Validação básica:** Utilizar campos obrigatórios (`required`) e os tipos de input apropriados nos formulários.
-

Desafios Extras (Opcionais)

1. **Resumo da Estante:** Exibir cards no topo da página de listagem mostrando o total de livros, quantos foram concluídos e a soma total de páginas lidas.
2. **Confirmação de Exclusão:** Inserir confirmação com JavaScript (`onClick="return confirm('Deseja excluir este livro da sua estante?')"`).
3. **Classificação por Estrelas (Nota):** Adicionar um campo extra de nota (1 a 5 estrelas) para livros marcados como "Lido".